# COL215 Lab Assignment 4a

## Hamming Distance and Similarity Engine

ANSH : 2025CS51207 & Priyam Gupta : 2025CS11135

## 1. Design Decisions

We implemented an 8-bit Hamming-distance and similarity engine as a combinational circuit. We use two 8-bit inputs, A and B, to calculate the bitwise difference, count the differing bits, check whether the inputs are equal, and determine whether the Hamming distance is within a specified threshold. We divided the implementation into reusable structural modules connected by the top-level hamming_distance8 module.

### Module 1: full_adder

**Inputs:** a, b, cin
**Outputs**: sum, cout

We use the full adder to add three 1-bit inputs and produce a 1-bit sum and a 1-bit carry. We implement the module directly with Boolean equations:

```
sum  = a ^ b ^ cin
cout = (a & b) | (b & cin) | (a & cin)
```

We use the XOR network to generate the sum bit, while the carry output becomes 1 whenever at least two of the three inputs are 1. We use this module as a building block inside popcount8.

### Module 2: half_adder

**Inputs:** a, b
**Outputs**: sum, cout

We use the half adder to add two 1-bit inputs. Its Boolean equations are:

```
sum  = a ^ b
cout = a & b
```

We use the half adder in popcount8 wherever two bits of the same weight need to be added without a carry-in.

### Module 3: popcount8

**Input:** bits[7:0]
**Output**: count[3:0]

We use the popcount8 module to compute the number of 1s present in an 8-bit input vector. We implement the circuit structurally as an adder tree using full adders and half adders. We use a 4-bit output because the count can range from 0 to 8.

- Stage 1: We use three full/half adders to reduce the eight input bits into partial sum and carry signals.
- Stage 2: We combine the three weight-1 sum bits using a full adder.
- Stage 3: We combine the weight-2 carry signals using two half adders.
- Stage 4: We combine the two remaining weight-2 sum bits using a half adder.
- Stage 5: We combine the resulting weight-4 carry information using a full adder.

```
count = {c7, s7, s6, s3}
```

For example, when bits = 00101000, we have two 1s, so the output count is 0010. When we connect this module to A XOR B, the resulting count gives the Hamming distance between the two 8-bit inputs.

## Module 4: comparator8_eq

**Input:** diff[7:0]
**Output**: eq

We use the comparator8_eq module to check whether all eight bits of diff are zero. We invert each bit and combine the results using an AND-tree. Therefore, eq becomes 1 only when diff = 00000000. Since we produce diff using A XOR B, this is equivalent to checking whether A and B are identical.

## Module 5: comparator4

**Inputs:** A[3:0], B[3:0]
**Outputs**: eq, lt, gt

We use the comparator4 module to compare two 4-bit unsigned values. We first form the bitwise equality, greater-than, and less-than conditions. Our final equations give priority to the most significant differing bit, following the standard unsigned comparison rule.

```
eq = 1  when A = B
gt = 1  when A > B
lt = 1  when A < B
```

We use this comparator to compare the calculated Hamming distance with the input threshold.

## Module 6: hamming_distance8

We use hamming_distance8 as the top-level module of our design.

**Inputs**: A[7:0], B[7:0], threshold[3:0]
**Outputs**: difference[7:0], distance[3:0], equal, within_threshold

### Step 1: Calculate the difference

We XOR the two 8-bit vectors:

```
difference = A ^ B
```

An XOR output bit becomes 1 exactly at positions where the corresponding bits of A and B are different. For our representative case:

```
A = 10110100
B = 10011100
    --------
    00101000
```

The difference vector therefore contains two 1s, which gives us two differing bit positions.

### Step 2: Calculate Hamming distance

We pass the difference vector to popcount8. The number of 1s in the difference vector gives us the Hamming distance between A and B. For 00101000, we obtain a distance of 2.

### Step 3: Determine equality

We pass the same difference vector to comparator8_eq. A zero difference vector means that there are no differing positions, so we obtain equal = 1 exactly when A = B.

### Step 4: Compare the distance with the threshold

We compare the 4-bit distance against the threshold using comparator4. The final output is:

```
within_threshold = cmp_lt | cmp_eq
```

Thus, we obtain within_threshold = 1 when distance <= threshold and 0 when distance > threshold. For distance = 2 and threshold = 3, we obtain 1.

# 2. Simulation Snapshot

We verify our design using the tb_hamming_distance8 testbench. We first check a representative handout example and then perform exhaustive functional checking over the selected 8-bit input pairs and threshold values. For each case, we independently compute the expected difference, population count, equality result, and threshold condition, and compare them with the DUT outputs.

Representative simulation case:

A = 10110100
B = 10011100
threshold = 3

Expected outputs:

difference = 00101000
distance = 2
equal = 0
within_threshold = 1

We can see that the two input vectors differ at two bit positions, represented by the two 1s in the difference vector. Our popcount8 circuit therefore produces distance = 2. Because A and B are not identical, we obtain equal = 0. Since 2 <= 3, we obtain within_threshold = 1. This waveform verifies the main functional path that we implemented: XOR difference detection, structural population counting, equality detection, and threshold comparison.

## Simulation Waveform Snapshots

We captured the following screenshots from the Vivado behavioral simulation and they show the waveform during our exhaustive testbench sweep and the  waveform includes the DUT inputs and outputs, along with the testbench verification signals.

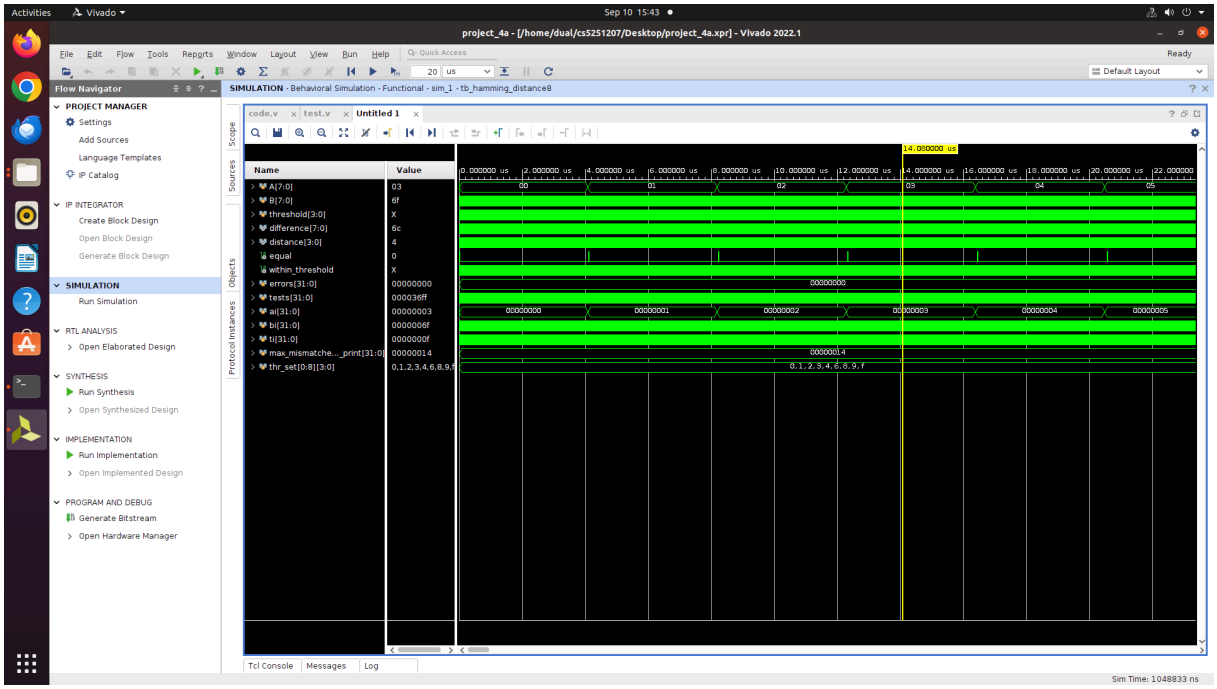



*Figure 1. Simulation waveform — early portion of the exhaustive verification run.*

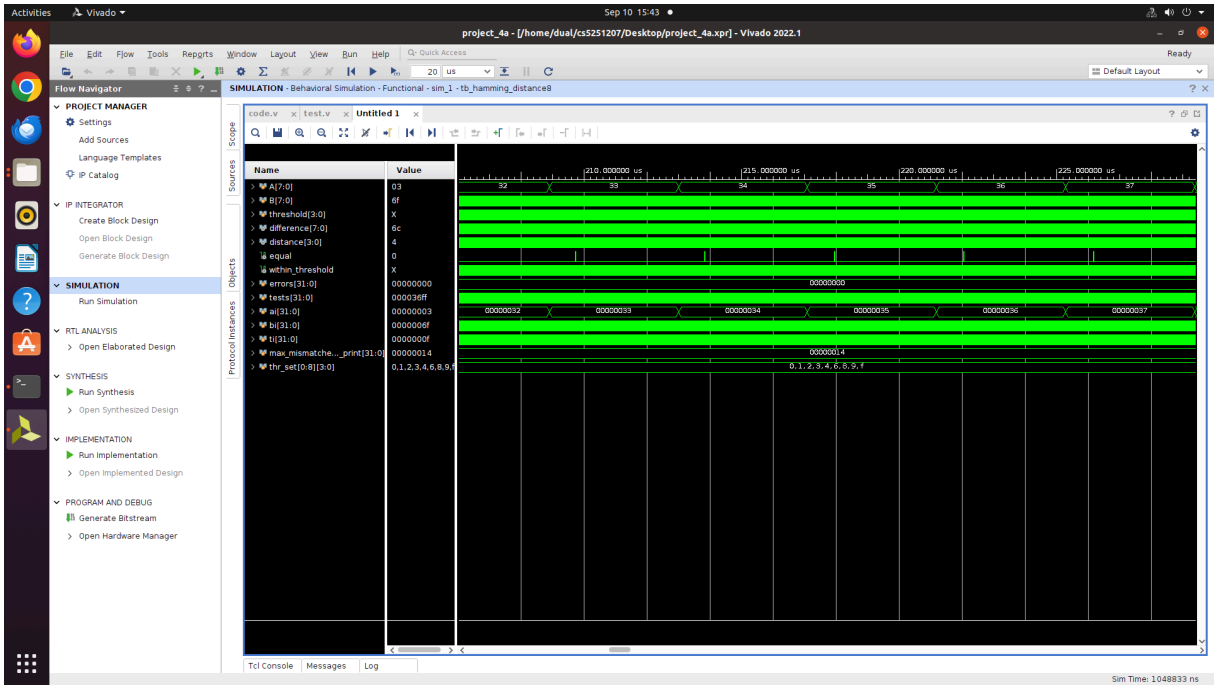



*Figure 2. Simulation waveform — continued exhaustive simulation activity.*

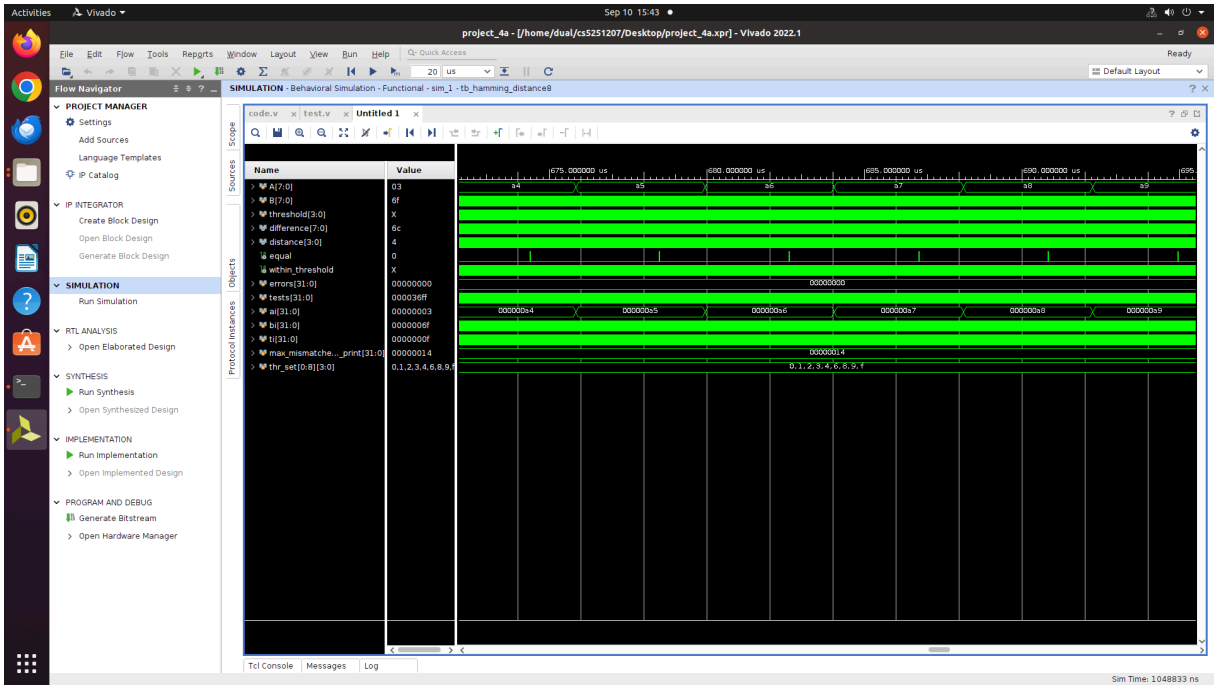



*Figure 3. Simulation waveform — further portion of the exhaustive simulation run.*

Note: These screenshots show different time windows of the same simulation run. Our testbench performs the exhaustive checking and prints the final PASS/FAIL result in the simulation console.